

**GamutX**

**JSON**

PYTHON SCRIPTING

# Introduction to JSON

- What is JSON ?
- Why Do We Need JSON?
- Why Save Data in JSON?
- Creating a JSON File
- Types of Data to Save in JSON
- Reading JSON Data

# What is JSON?

- **JSON stands for JavaScript Object Notation.**
- It is a file format used to store and exchange data.
- It looks very **similar to a Python dictionary**. It is:
  - Lightweight
  - Easy to read for humans
  - Easy to parse for machines



## sample Dictionary

```
data = {  
    "name": "Shot01",  
    "frame_range": [100, 200]  
}
```

## Where is JSON Used in VFX?

- Commonly used for **configuration files**.
- Helps store important settings for:
  - **Tools**
  - **Scripts**
  - **Pipelines**
- Useful when working across **multiple machines** in a studio.

## sample JSON

```
{  
    "name": "Shot01",  
    "frame_range": [100, 200]  
}
```

# How is Data Stored in JSON?

- JSON stores data as **key-value pairs**.
- The key is always a **string in double quotes**.
- The key is followed by a colon (:) and then the value.

## Example JSON Format:

```
{  
  "key": "value",  
  "another_key": "another_value"  
}
```

key

value

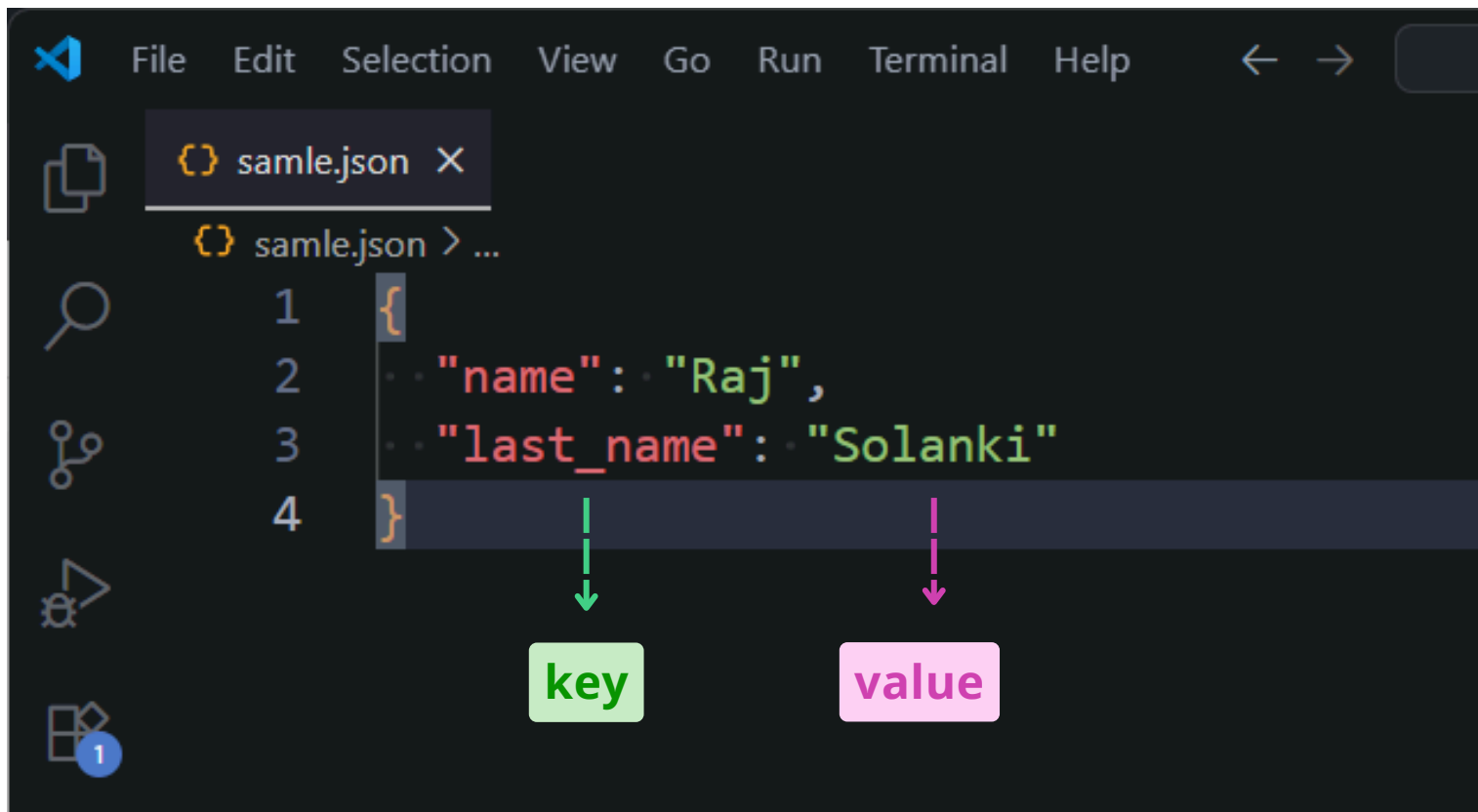
Separated by colon



# Creating a JSON File

Let's demonstrate how to create a JSON file.

1. Open any text editor (e.g., Notepad, VS Code).
2. Type the following content:



```
File Edit Selection View Go Run Terminal Help
{ } samle.json x
{ } samle.json > ...
1 {
2   "name": "Raj",
3   "last_name": "Solanki"
4 }
```

key value

3. Save the file as: sample.json

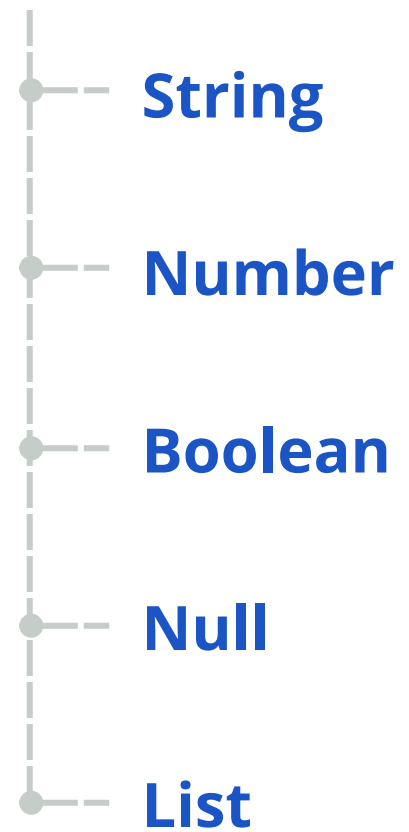


## Note:

- Always use **.json** as the file extension.

# Types of Data to Save in JSON

- A **JSON** (JavaScript Object Notation) file can contain various **types** of **data** each represented in a **specific format**.
- The primary data types that JSON supports include:



## Common JSON Data Types

### 1. String

Text data enclosed in double quotes.

```
{
  "name": "John Doe",
  "city": "New York"
}
```

String

### 2. Number

Can be an integer or decimal (floating-point).

```
{
  "age": 30,
  "height": 5.9
}
```

Number

### 3. Boolean

- Represents true or false value
- Useful for
  - toggles, switches, or flags

```
{
  "isStudent": true,
  "isEmployed": false
}
```

Boolean

### 4. Null

Represents an empty or undefined value.

```
{
  "middleName": null
}
```

Null

### 5. List (Array)

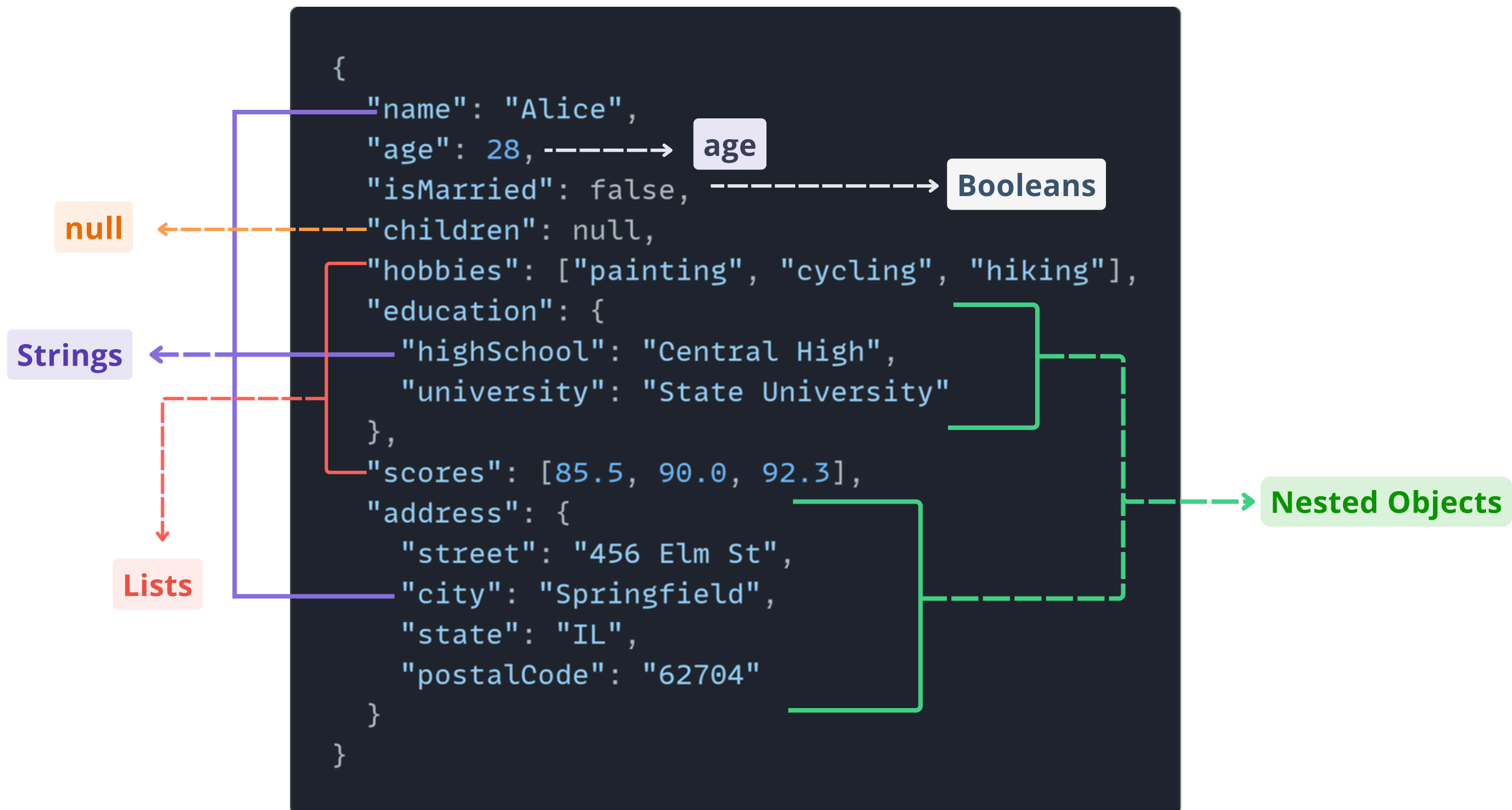
- A collection of values inside square brackets [ ].
- Can contain strings, numbers, or even other objects.

```
{
  "favoriteColors": ["red", "green", "blue"],
  "scores": [95, 88, 76]
}
```

List (Array)

## Complete Example with All Data Types

Here's a full example that includes **all the major JSON data types** in one structure:



# Reading JSON Data

- Instead of putting data directly in our **script (hardcoding)**, we can **store it in a separate file**.
- To read Data we **used `json.load()`**

## Example JSON File (sample.json)

Suppose we have a file named sample.json

```
{  
  "name": "Raj",  
  "last_name": "Solanki"  
}
```

key

Value

main.py file

```
import json -----> Module  
  
with open("sample.json", 'r') as f:  
    data = json.load(f)  
  
print(data)  
  
# Output:  
# {'name': 'Raj', 'last_name': 'Solanki'}
```

- Reads data from a **file object** and **converts it into a Python dictionary or list**.



## Common JSON Operations

- Nested JSON
- Accessing JSON Data
- Adding New Data to JSON
- Writing JSON Data
- Writing JSON Data - in a Prettier Format
- Modifying Existing JSON Data
- Handling Errors and Exceptions
- Best Practices for JSON Handling
- Find Sample JSON data online

# Nested JSON

JSON data can be nested,

- **an object can contain another object** or **array**, and arrays can contain objects or other arrays.

## Example of Nested JSON:

```
{
  "store": {
    "book": [
      {
        "category": "fiction",
        "title": "The Great Gatsby",
        "author": "F. Scott Fitzgerald",
        "price": 10.99
      },
      {
        "category": "non-fiction",
        "title": "Sapiens",
        "author": "Yuval Noah Harari",
        "price": 15.99
      }
    ],
    "bicycle": {
      "color": "red",
      "price": 399.99
    }
  }
}
```

Top-level key → holds an object.

- Inside "store", there are **two keys**:
  - "book" contains a **list of objects** (each one is a book).
  - "bicycle" contains another object with details about a bicycle.

# Accessing JSON Data

Now let's see how we can **read a JSON file** and **access specific data** from it using Python.

## Access the First Person's Data

### Sample JSON File: sample.json

index

0

1

2

```
{
  "people": [
    {
      "name": "aman",
      "last_name": "verma",
      "location": ["mumbai", "banglore"]
    },
    {
      "name": "sumit",
      "last_name": "k",
      "location": ["mumbai"]
    },
    {
      "name": "manoj",
      "last_name": "dombale",
      "location": ["mumbai", "banglore", "hyd"]
    }
  ]
}
```

### Python Code:

```
import json

with open("sample.json", 'r') as f:
    data = json.load(f)

print(data["people"][0])
```

read

Access 0<sup>th</sup> index

### Output:

```
{
  "name": "aman",
  "last_name": "verma",
  "location": ["mumbai", "banglore"]
}
```



## Get the last\_name of Every Person

- We can **loop through the list** and **access particular fields** → like "**last\_name**" for **each person**.

### Sample.json

```
{
  "people": [
    {
      "name": "aman",
      "last_name": "verma",
      "location": ["mumbai", "banglore"]
    },
    {
      "name": "sumit",
      "last_name": "k",
      "location": ["mumbai"]
    },
    {
      "name": "manoj",
      "last_name": "dombale",
      "location": ["mumbai", "banglore", "hyd"]
    }
  ]
}
```

### Python Code:

```
import json

with open("sample.json", 'r') as f:
    data = json.load(f)

for each in data["people"]:
    print(each["last_name"])
```

Output:

```
verma
k
dombale
```

### How This Works:

- We looped over the list in **data["people"]**.
- For each person (a dictionary), we accessed the "**last\_name**" key.

# Writing JSON Data

Now let's see how to **create a new JSON file** and **add data to it** using Python.

- The **json.dump() method writes a Python dictionary or list to a file in JSON format.**

## Example 1: Writing a Dictionary to a File

### Python Code: script.py

```
import json

data = {
    "name": "Alice",
    "age": 28,
    "is_student": False,
    "courses": ["Art", "Design"]
}

with open('output.json', 'w') as file:
    json.dump(data, file)
    .....
```

### Output in output.json

```
{"name": "Alice", "age": 28, "is_student": false, "courses": ["Art", "Design"]}
```

Write Python Dictionary into JSON Format

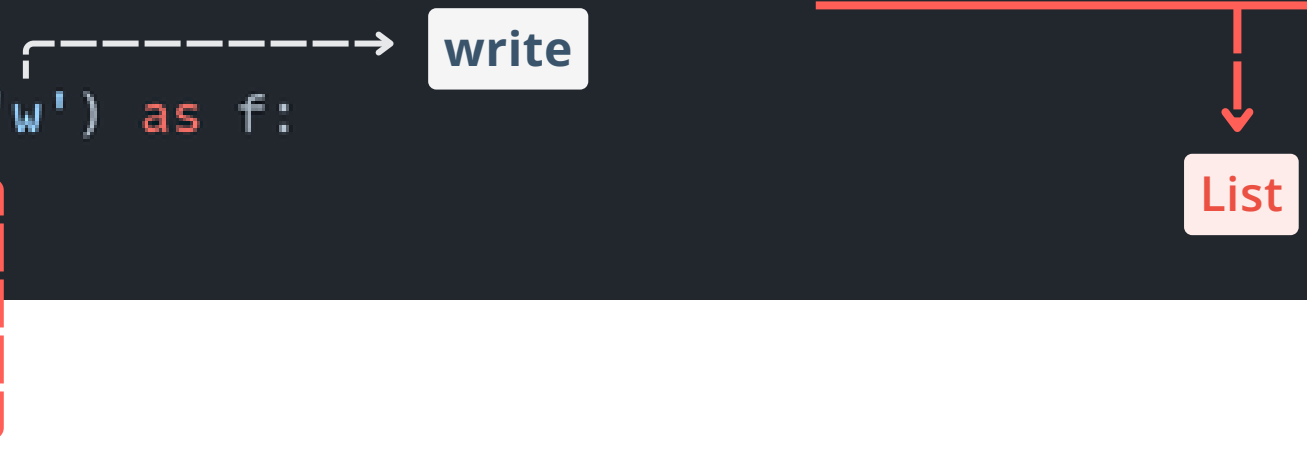
The data is **converted to JSON format** before being saved.

## Example 2: Another Dictionary with List

```
import json

data = {'name': 'aman', 'last_name': 'verma', 'location': ['mumbai', 'banglore']}

with open("sample_output.json", 'w') as f:
    json.dump(data, f)
```

Output in **sample\_output.json**

```
{"name": "aman", "last_name": "verma", "location": ["mumbai", "banglore"]}
```

# Writing JSON Data – in a Prettier Format

- We can save JSON data in a more readable, pretty-printed format
  - using the **indent argument in the json.dump() method**.

## Python Code script.py

```
import json

data = {
    'name': 'aman',
    'last_name': 'verma',
    'location': ['mumbai', 'banglore']
}

with open("sample_output.json", 'w') as f:
    json.dump(data, f, indent=4)
```

-----→ use for pretty printing

## Output in sample\_output.json

```
{
    "name": "aman",
    "last_name": "verma",
    "location": [
        "mumbai",
        "banglore" ]
}
```

## Benefits of Pretty Printing

- Easier for **humans to read and debug**
- Helpful when **sharing config or data files with team members**
- **Better for version control (Git)** as **changes are easier to spot**

# Modifying Existing JSON Data

- We can **modify** the data inside an existing JSON file. → let's **change** the **last\_name** of the first person.

Sample JSON File: **sample.json**

index

```
{
  "people": [
    {
      "name": "aman",
      "last_name": "verma",
      "location": ["mumbai", "bangalore"]
    },
    {
      "name": "sumit",
      "last_name": "k",
      "location": ["mumbai"]
    },
    {
      "name": "manoj",
      "last_name": "dumble",
      "location": ["mumbai", "bangalore", "hyd"]
    }
  ]
}
```

0

1

2

change to sharma

Python Code: **script.py**

```
import json -----> Module

with open("sample.json", 'r') as f:
    data = json.load(f)

data["people"][0]["last_name"] = "sharma"

with open("sample.json", 'w') as f:
    json.dump(data, f, indent=4)

print(data)
```



Output **sample.json**

```
{
  "people": [
    {
      "name": "aman",
      "last_name": "sharma",
      "location": [
        "mumbai",
        "banglore"
      ]
    },
    {
      "name": "sumit",
      "last_name": "k",
      "location": [
        "mumbai"
      ]
    },
    {
      "name": "manoj",
      "last_name": "domble",
      "location": [
        "mumbai",
        "banglore",
        "hyd"
      ]
    }
  ]
}
```

Using Python's json module, we can easily:

- **Read** → data from a JSON file using json.load().
- **Write** → data to a JSON file using json.dump().
- **Modify** → data by accessing dictionary keys and list indices.
- **Pretty-print the data** → with indentation for better readability.

# Handling Errors and Exceptions in JSON

## 1. Handling Invalid JSON Strings

- **Problem:** JSON string has a **formatting error** (e.g., **extra comma, Missing Brackets**).

```
import json

invalid_json_string = '{"name": "Alice", "age": 30,}'

try:
    data = json.loads(invalid_json_string)
except json.JSONDecodeError as e:
    print(f"JSON decode error: {e}")
```

Having comma → leads to Syntax Error

- Python raises a **json.JSONDecodeError**.

**Output:**

```
JSON decode error: Expecting property name enclosed in double quotes: line 1 column 31 (char 30)
```



## 2. Handling Missing Files

- **Problem:** Trying to open a file that doesn't exist
- **Example:** FileNotFoundError
- **Solution:** Use try-except to catch the error and show a message

```
import json

try:
    with open('non_existent_file.json', 'r') as file:
        data = json.load(file)
except FileNotFoundError as e:
    print(f"File not found: {e}")
```

Not Exist

### Folder Structure

project/

├── main.py

├── users.json

✅ Existing file

Output:

```
File not found: [Errno 2] No such file or directory: 'non_existent_file.json'
```



# Best Practices for JSON Handling

## 1. Use Indentation for Readability

When writing JSON to a file, use `indent=4` with `json.dump()` or `json.dumps()`.

**Example:**

```
json.dump(data, file, indent=4)
```

**Why?**

- **Easier** to read
- **Better** for debugging
- **Cleaner** version control diffs

## 2. Handle Invalid JSON Gracefully

Always wrap **JSON loading** in a **try-except** block to **catch bad data**.

**Example:**

```
try:  
    data = json.loads(json_data)  
except json.JSONDecodeError as e:  
    print(f"Invalid JSON: {e}")
```

**Why?**

- **Prevents** script **from crashing** if the input is malformed
- Lets us log or **show** a helpful **error message**

### 3. Check the Data Type After Loading

- Use **type()** or **isinstance()** to verify loaded JSON

#### Example:

```
data = json.loads(json_data)

if isinstance(data, dict):
    print("We loaded a dictionary.")
elif isinstance(data, list):
    print("We loaded a list.")
```

Check if it's a dictionary

Check if it's a list

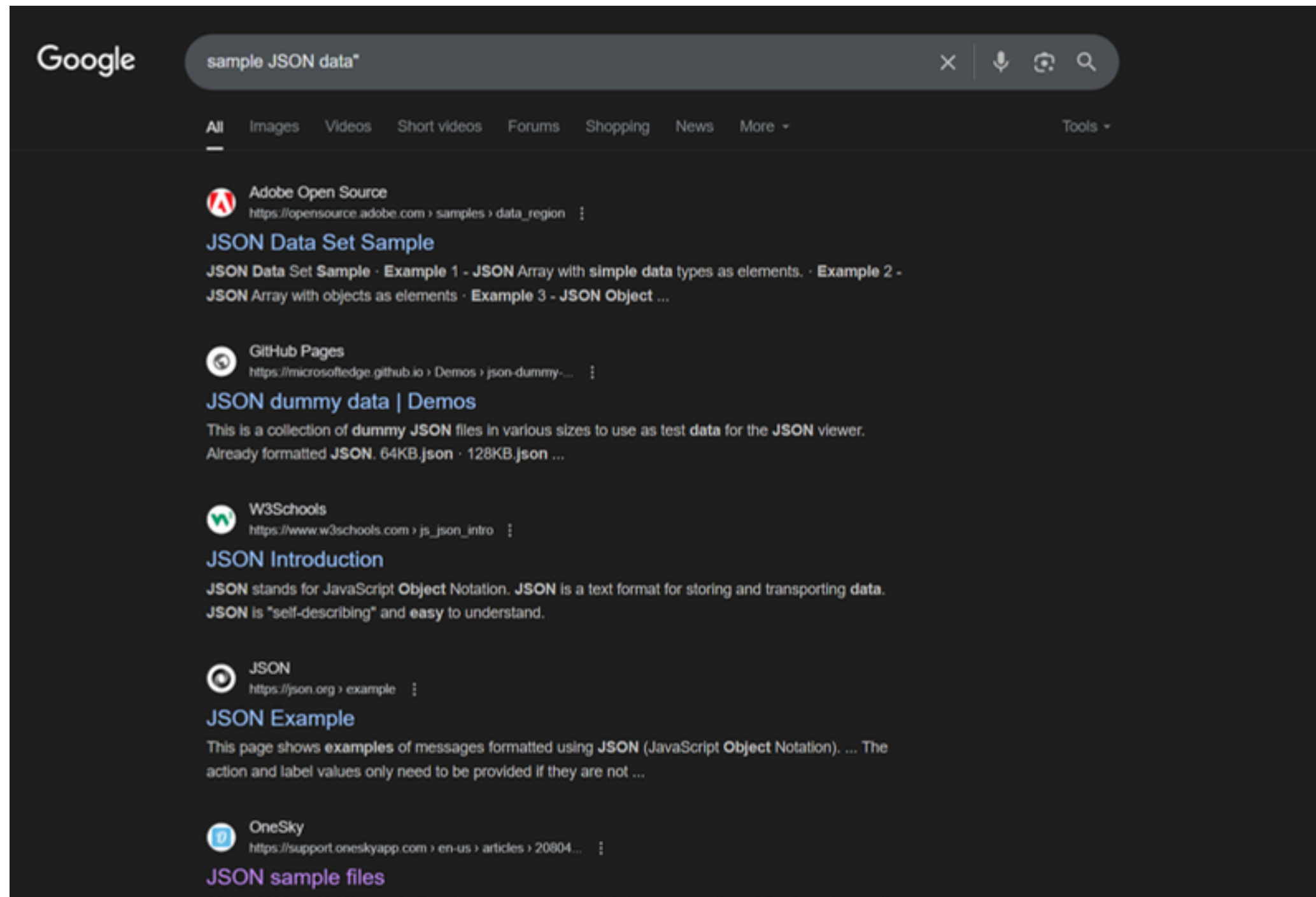
#### Why?

- **JSON** can be a **dict**, **list**, **string**, **number**, etc.
- This **Prevents** type-related **bugs**

# Finding Sample JSON Data

To get started with JSON, we can either **find sample JSON data online or create your own JSON data.**

## Option 1: Search for Sample JSON Data Online



### Steps:

- A quick Google search like:  
*"sample JSON data"*
- will show you lots of examples.

### Popular websites for free JSON data:

- [JSON Placeholder](#)
- [Mockaroo](#)
- [DummyJSON](#)
- [JSON Data Generator](#)

## Why Use Sample JSON Data?

- Helps you understand **how real-world data** is structured.
- Great for practicing:
  - **Reading/parsing JSON**
  - **Looping through arrays**
  - **Updating or saving modified data**

### These sites offer:

- Sample **user profiles**, **product catalogs**, **articles**, etc.
- You can **download the JSON** or **copy-paste** into your project.

```
{
  "shot_id": "SH020",
  "frame_range": [1001, 1050],
  "camera": "cam_final",
  "resolution": [1920, 1080],
  "render_settings": {
    "renderer": "Arnold",
    "format": "exr",
    "samples": 4
  },
  "assets": [
    {
      "name": "hero_robot",
      "type": "model",
      "version": "v008"
    }
  ],
  "published_by": "alex.w",
  "timestamp": "2025-07-01T10:30:00Z"
}
```

## Using JSON Data in VS Code

We can easily write our own JSON data right in VS Code or any code editor.

### Steps:

1. Open VS Code
2. Create a file named mydata.json
3. Add key-value pairs like:

### Example:

sample.json

```
{  
  "name": "John",  
  "age": 30,  
  "location": "New York"  
}
```



### 💡 Use It in Python:

```
import json  
  
with open('mydata.json', 'r') as file:  
    data = json.load(file)  
    print("Original Data:", data)  
  
data['age'] = 31  
data['location'] = 'San Francisco'  
  
with open('mydata.json', 'w') as file:  
    json.dump(data, file, indent=4)  
    print("Updated data saved.")
```

Load

Modify

Save



# Types of JSON Data Objects (JSON Data Handling)

- Types of JSON Data Objects
- Only List
- Only Dict
- List of List
- List of Dict
- Dict of List
- Dict of Dict

# Types of JSON Data Objects

- JSON data comes in various shapes—**simple, nested, Complex**
- Understanding data structure is **key** to **reading, modifying, or writing JSON**.

## Common JSON Data Types:

| Types                  | Examples                                           |
|------------------------|----------------------------------------------------|
| • <b>Only List</b>     | → ["apple", "banana"]                              |
| • <b>Only Dict</b>     | → {"name": "Alice", "age": 30}                     |
| • <b>List of Lists</b> | → [[1, 2], [3, 4]]                                 |
| • <b>List of Dicts</b> | → [{"id": 1}, {"id": 2}]                           |
| • <b>Dict of Lists</b> | → {"fruits": ["apple", "banana"]}                  |
| • <b>Dict of Dicts</b> | → {"user1": {"name": "A"}, "user2": {"name": "B"}} |



# Only List

- **Used** when **JSON data** is just a **list**
- This is **very common** when dealing **with collections**—like a list of **names, fruits, or file paths**.

JSON File: **only\_list.json**

```
[  
  "Apple",  
  "Banana",  
  "Cherry"  
]
```

→ List of Strings

Python Code: **Reading the List**

```
import json  
  
with open("only_list.json", "r") as file:  
    data = json.load(file)  
  
for item in data:  
    print(item)
```

**Output:**

```
Apple  
Banana  
Cherry
```

🧠 **What Happens:**

- `json.load()` loads the list
- `for` loop prints each item



# Only Dict

- Used when JSON **stores a single record** with key-value pairs — like user **info** or **settings**.

JSON File: **only\_dict.json**

```
{  
  "name": "John",  
  "age": 30,  
  "city": "New York"  
}
```

Only Dict

Collection of Key Value Pairs

Python Code: **Reading the Dictionary**

```
import json  
  
with open("only_dict.json") as file:  
    data = json.load(file)  
  
print(data["name"]) # John  
print(data["age"])  # 30  
print(data["city"]) # New York
```

## 🧠 What Happens:

- json.load() gives a Python dict
- Then we can Access values using keys

# List of List

- Used for matrix-like data, grids, or grouped values.

JSON File: list\_of\_list.json

`[[1, 2, 3], [4, 5, 6], [7, 8, 9]]` → Main list

Inner List →

List 1

List 2

List 3

Python Code: Looping Through Nested Lists

```
import json

with open("list_of_list.json", "r") as file:
    data = json.load(file)

for sublist in data:
    for item in sublist:
        print(item)
```

Output:

1  
2  
3  
4  
5  
6  
7  
8  
9

🧠 How it Works:

- data is a **list of lists**
- **First loop** = each sublist
- **Second loop** = each item inside

## Nested Arrays of Different Types

- This is a **JSON array** of arrays, with **each nested array** containing :

```
[  
  [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
  ],  
  [  
    ["a", "b", "c"],  
    ["d", "e", "f"],  
    ["g", "h", "i"]  
  ],  
  [  
    [true, false, true],  
    [false, true, false],  
    [true, true, false]  
  ]  
]
```

Numbers list

String List

Boolean List

# List of Dict

- **Used** when **storing multiple records**, each with its own details —
  - like **employee data** or **configs**.

📁 JSON File: **list\_of\_dict.json**

```
[  
  {"id": 1, "name": "Alice"},  
  {"id": 2, "name": "Bob"}  
]
```

→ Each Element → Dictionary

Python Code: **Reading and Accessing the Data**

```
import json  
  
with open("list_of_dict.json", "r") as file:  
    data = json.load(file)  
  
for item in data:  
    print(f"ID: {item['id']}, Name: {item['name']}")
```

Output: ←

```
ID: 1, Name: Alice  
ID: 2, Name: Bob
```

## Example — Employee Records

- A **JSON array** of **objects**, with each object represents an **employee** with **attributes contained in an array**.

```
[  
  {"name": "Alice", "department": "Engineering", "salary": 70000},  
  {"name": "Bob", "department": "HR", "salary": 50000},  
  {"name": "Charlie", "department": "Engineering", "salary": 80000},  
  {"name": "David", "department": "Marketing", "salary": 60000},  
  {"name": "Eve", "department": "HR", "salary": 55000}  
]
```

Dict

List

### What This Represents:

- Each item is an **employee record**.
- Each record is a **dictionary** with:
  - **"name"**
  - **"department"**
  - **"salary"**



# Dict of List

- In this structure, your **JSON object contains keys**, and **each key maps to a list of values**.
- **useful** when **grouping** related items—for example, **grouping fruits, vegetables**, or **subjects**.

## Dict of List (Grouped Items)

JSON File: **dict\_of\_list.json**

```
{  
  "fruits": ["apple", "banana"],  
  "vegetables": ["carrot", "spinach"]  
}
```

Each Key Points to List of values

Python Code: **Reading and Using the Data**

```
import json  
  
with open("dict_of_list.json", "r") as file:  
    data = json.load(file)  
  
print("Fruits:", data["fruits"])  
print("Vegetables:", data["vegetables"])  
  
for fruit in data["fruits"]:  
    print("Fruit:", fruit)  
  
for vegetable in data["vegetables"]:  
    print("Vegetable:", vegetable)
```

Output:

```
Fruits: ['apple', 'banana']  
Vegetables: ['carrot', 'spinach']  
Fruit: apple  
Fruit: banana  
Vegetable: carrot  
Vegetable: spinach
```

## Nested Dict of Lists

### JSON File:

```
{  
  "subjects": {  
    "math": ["algebra", "geometry", "calculus"],  
    "science": ["physics", "chemistry", "biology"],  
    "languages": ["English", "Spanish", "French"]  
  }  
}
```

Top level Key

Dict of List

### Python Code: Looping Through Nested Data

```
import json  
  
with open("nested_dict_of_list.json", "r") as file:  
    data = json.load(file)  
  
subjects = data["subjects"]  
    Key      Value  
  
for category, topics in subjects.items():  
    print(f"\nCategory: {category}")  
    for topic in topics:  
        print(" -", topic)
```

### Output:

```
Category: math  
- algebra  
- geometry  
- calculus  
  
Category: science  
- physics  
- chemistry  
- biology  
  
Category: languages  
- English  
- Spanish  
- French
```

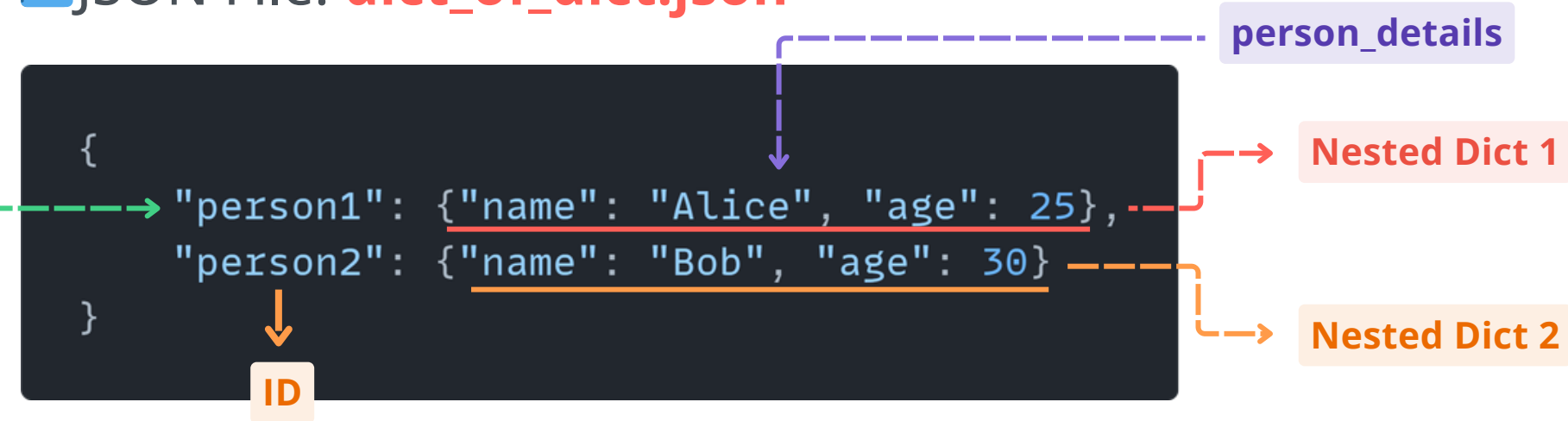
Key

List

# Dict of Dict

- Used to represent hierarchical or structured data like users, configs, or org charts.

JSON File: **dict\_of\_dict.json**



Python Code: **Reading Nested Dictionaries**

```
import json  
  
with open("dict_of_dict.json", "r") as file:  
    data = json.load(file)  
  
for person_id, person_details in data.items():  
    print(f"{person_id}: Name - {person_details['name']}, Age - {person_details['age']}")
```

Annotations:

- A blue arrow points from the **Top-level key** label to the `person_id` variable in the loop.
- A green arrow points from the **Nested dict access** label to the `person_details['name']` and `person_details['age']` expressions in the print statement.

## Output

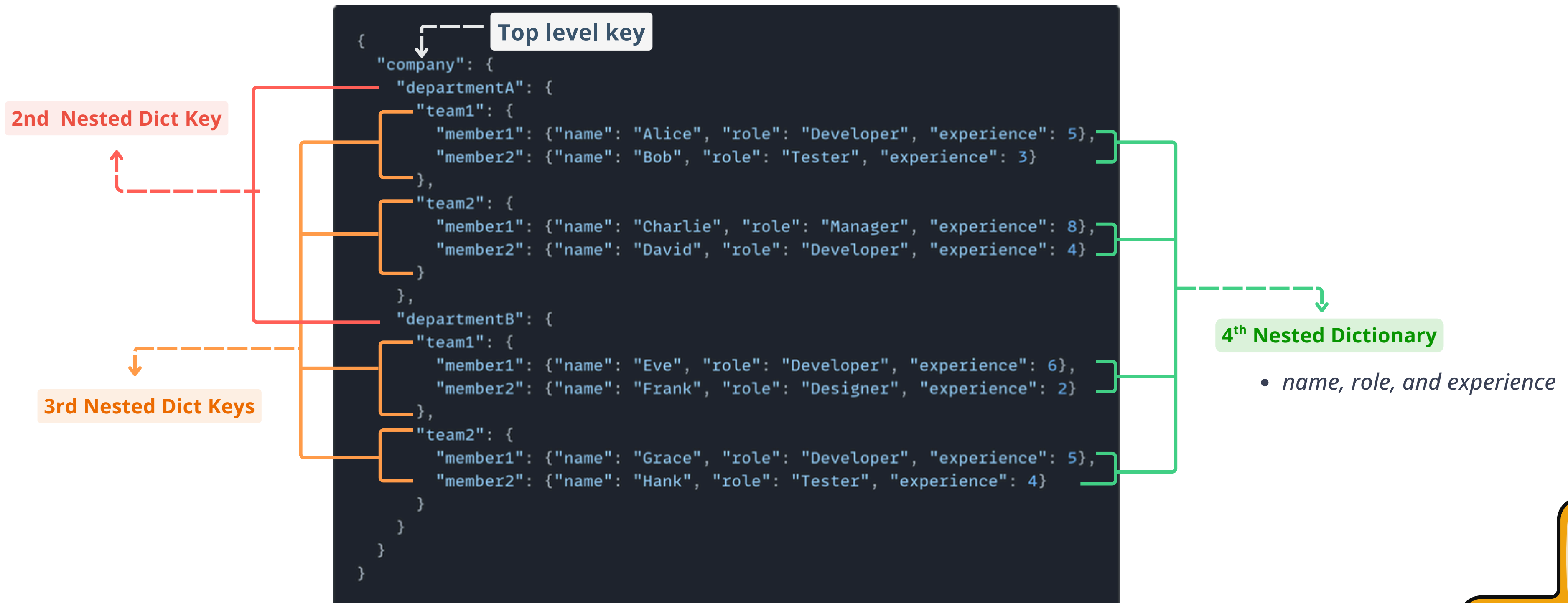
```
person1: Name - Alice, Age - 25  
person2: Name - Bob, Age - 30
```



## Example - Deeply Nested Data

- Let's look at a deeper example that represents a company with multiple **departments, teams, and members**.

JSON File (**Nested Dict of Dict**)



# Summary Table (File-Based)

| JSON Type      | Example Structure                 | Python Access Method                  | Use Case                                        |
|----------------|-----------------------------------|---------------------------------------|-------------------------------------------------|
| • Only List    | → ["apple", "banana", "cherry"]   | → Iterate over the list.              | → Simple arrays, IDs, or tags                   |
| • Only Dict    | → {"name": "John", "age": 30}     | → Access values using keys.           | → Single entity with attributes                 |
| • List of List | → [[1, 2, 3], [4, 5, 6]]          | → Nested loops to access items.       | → Grid-like data                                |
| • List of Dict | → [{"id": 1, "name": "Alice"}]    | → Loop through list, access dict keys | → Multiple entities with attributes             |
| • Dict of List | → {"fruits": ["apple", "banana"]} | → Access lists by key, then iterate   | → Grouped collections (e.g., fruits/vegetables) |
| • Dict of Dict | → {"person1": {"name": "Alice"}}  | → Loop through dict, then nested keys | → Hierarchical or nested attributes             |

## Parsing JSON Data

- Parsing JSON Strings to Dictionaries - `json.loads`
- Parsing Dictionaries to JSON Strings - `json.dumps`

# Parsing JSON Strings to Dictionaries - json.loads

Use `json.loads()` to convert a JSON string into a **Python object**.

## Convert JSON String to Dictionary

```
import json

json_string = '{"name": "Bob", "age": 25, "city": "Builderland"}'

python_dict = json.loads(json_string)

print(python_dict)
```

Dictionary

Output:

```
{'name': 'Bob', 'age': 25, 'city': 'Builderland'}
```

## Convert JSON String to List

```
json_string_list = '["apple", "banana", "cherry"]'

python_list = json.loads(json_string_list)

print(python_list)
```

List

Output:

```
['apple', 'banana', 'cherry']
```

# Parsing Dictionaries to JSON Strings - json.dumps

Use `json.dumps()` to **convert** a **Python object** (dictionaries or lists) into a **JSON-formatted string**.

This is useful for:

- **Logging** or **displaying data**
- Sending **data over a network**
- **Saving configuration** as a **string**

## Convert Dictionary to JSON String

```
import json
```

```
python_dict = {  
    "name": "Bob",  
    "age": 25,  
    "city": "Builderland"  
}
```

Dictionary

This Convert Dict to String

```
json_string = json.dumps(python_dict)  
print(json_string)  
print(type(json_string))
```

Output:

```
{"name": "Bob", "age": 25, "city": "Builderland"}  
<class 'str'>
```



## Convert List to JSON String

```
python_list = ["apple", "banana", "cherry"]  
json_string_list = json.dumps(python_list)  
print(json_string_list)
```

Diagram: A purple dashed arrow points from the word "List" in a purple box to the `python_list` variable. A pink dashed arrow points from the `print` statement to the output box.

List

Output:

```
["apple", "banana", "cherry"]
```

### Key Notes:

- **Loading JSON Data**
  - `json.load(file)` → Read JSON Dict from a file
  - `json.loads(string)` → Parse JSON Dict from a string
- **Dumping JSON Data**
  - `json.dump(data, file, indent=4)` → Write Dict to a file
  - `json.dumps(data, indent=4)` → Convert Dict to a JSON string

✓ Use `indent=4` for readable, pretty formatting

## JSON in VFX Pipeline

- Configuration Files for Software and Tools
- Asset Management
- Pipeline Management
- Team Communication and Collaboration

# Configuration Files for Software and Tools

In a VFX studio, different software tools are used for different tasks:

- Maya → **animation**
- Nuke → **compositing**
- Arnold & RenderMan → **rendering**
- Each of these tools needs specific **settings** to run properly—like **resolution, file format, or lighting setup**.
- **Studios use JSON files** to store and share these settings automatically.

## Example: Render Settings JSON

Example: **render\_settings.json**

```
{
  "render_settings": {
    "resolution": "1920x1080",
    "output_format": "PNG",
    "quality": "high",
    "lighting": "studio_lighting"
  }
}
```

A script can *read this JSON file to ensure all machines use the same render settings.*



# Asset Management

- Assets like **models**, **textures**, **rigs**, and **animations** are shared across projects.
- **Why Use JSON?**
  - To track asset details like file paths, types, and metadata.

## Example: Asset Tracking

```
{
  "assets": [
    {
      "id": 1,
      "name": "city_model",
      "type": "3D_model",
      "path": "/assets/3d/city.obj"
    },
    {
      "id": 2,
      "name": "character_rig",
      "type": "rig",
      "path": "/assets/rigs/character_rig.fbx"
    }
  ]
}
```



### Purpose:

- Helps departments like **modeling**, **animation**, and **lighting** easily locate and manage resources.

# Pipeline Management

- In VFX, a **pipeline** is the **step-by-step** process from **scene setup** to **final render**.
- **Studios** use **scripts** and **JSON configs** to **automate** and **manage** these steps efficiently.

## Example: JSON for Pipeline Steps

```
{
  "pipeline_steps": [
    {
      "step": "scene_setup",
      "tool": "Maya",
      "command": "maya -batch -file scene.mb"
    },
    {
      "step": "rendering",
      "tool": "Arnold",
      "command": "render --camera mainCamera --resolution 1920x1080"
    }
  ]
}
```

Diagram annotations:

- A green box labeled "Step name" with a green arrow pointing to the "step" field of the first object.
- A white box labeled "tool to use" with a dashed white arrow pointing to the "tool" field of the first object.
- A red box labeled "command to run" with a red arrow pointing to the "command" field of the first object.

## Why Use JSON for Pipeline Steps?

- Defines each pipeline step clearly
- Easy to automate and update
- Departments (**modeling, lighting, FX**, etc.) can access only what they need

# Team Communication and Collaboration

- In large VFX studios, teams share feedback and status updates on **shots** and **assets**.
- JSON Used → Stores **comments**, **approvals**, and **shot status** in a structured way.

## Example: Feedback Management

```
{
  "shots": [
    {
      "shot_id": "A001",
      "feedback": "Fix lighting on the character.",
      "status": "approved"
    },
    {
      "shot_id": "A002",
      "feedback": "Add more reflections to the glass surface.",
      "status": "pending"
    }
  ]
}
```

### Why Use JSON?

- **Easy to share** across teams
- Script-friendly for **automation**
- Keeps feedback **organized** and **searchable**

# Why Do We Need JSON?

## 1. Sharing Data Across Scripts and Applications

- In VFX, one task often involves **multiple tools or scripts**.
- These tools need to **exchange data** — JSON helps with that.

### Why JSON works well:

- It organizes data in a simple, readable format.
- It can be used by tools written in different programming languages.

### Exapmle:

- A **render manager** saves shot details (like frame ranges) to a JSON file.

```
{
  "shot_id": "SH010",
  "frame_range": [1001, 1100],
  "camera": "cam_main",
  "resolution": [1920, 1080],
  "output_path": "/projects/ep01/shots/SH010/render/v001/"
}
```

## 2. Storing Configuration Settings

- JSON is commonly used to store tool or project settings. **Common settings Like**

```
{
  "project_name": "DragonHunt",
  "default_resolution": [2048, 858],
  "plugin_paths": [
    "/mnt/plugins/maya",
    "/mnt/plugins/nuke"
  ],
  "render_settings": {
    "quality": "high",
    "format": "exr",
    "motion_blur": true
  }
}
```



### 3. Reusability and Portability

- JSON data can be saved once and **reused** in:
  - Different Python scripts
  - Other tools like Maya, Nuke, or Houdini

```
{  
  "asset_name": "dragon_wings",  
  "version": "v003",  
  "published_by": "jane.doe",  
  "tags": ["creature", "wing", "rigged"],  
  "file_path": "/assets/creatures/dragon/wings/v003/dragon_wings.ma"  
}
```

#### Why it's portable:

- JSON is supported by almost all programming languages.
- Makes it easy to **move data between tools** or even between departments.

# **Concept Practice Task**

- Common JSON Operations
- Types of JSON Data Object

## Save user input to a JSON file

```
import json

# Prompt user for input
name = input("Enter your name: ")
city = input("Enter your city: ")

# Create a dictionary
user_data = {
    "name": name,
    "city": city
}

# Save data to a JSON file
with open("user_data.json", "w") as f:
    json.dump(user_data, f, indent=4)

print("User data saved to user_data.json.")
```

Resulting user\_data.json:

```
{
    "name": "Alice",
    "city": "New York"
}
```

## Pretty-print a JSON file

Required sample.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    },
    {
      "name": "Bob",
      "last_name": "Johnson",
      "location": ["San Francisco", "Los Angeles"]
    }
  ]
}
```

Main Python Code:

```
import json

# Load a JSON file
with open("sample.json", "r") as f:
    data = json.load(f)

# Pretty-print JSON data
print(json.dumps(data, indent=4))
```



## Check if a key exists in JSON

Required sample.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    },
    {
      "name": "Bob",
      "last_name": "Johnson",
      "location": ["San Francisco", "Los Angeles"]
    }
  ]
}
```

Main Python Code:

```
import json

# Load JSON data
with open("sample.json", "r") as f:
    data = json.load(f)

# Check if a key exists in the first person's data
key_to_check = "last_name"
if key_to_check in data["people"][0]:
    print(f"Key '{key_to_check}' exists with value: {data['people'][0][key_to_check]}")
else:
    print(f"Key '{key_to_check}' does not exist.")
```

## Count occurrences in JSON data

Required sample.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    },
    {
      "name": "Bob",
      "last_name": "Johnson",
      "location": ["San Francisco", "New York"]
    }
  ]
}
```

Main Python Code:

```
import json

# Load JSON data
with open("sample.json", "r") as f:
    data = json.load(f)

# Count occurrences of a specific city
city_to_count = "New York"
count = sum(city_to_count in person["location"] for person in data["people"])

print(f"'{city_to_count}' appears in {count} locations.")
```

## Extract specific data from nested JSON

Required sample.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    },
    {
      "name": "Bob",
      "last_name": "Johnson",
      "location": ["San Francisco", "Los Angeles"]
    }
  ]
}
```

Main Python Code:

```
import json

# Load JSON data
with open("sample.json", "r") as f:
    data = json.load(f)

# Extract all cities from the "location" field
locations = [loc for person in data["people"] for loc in person["location"]]
print("Locations:", locations)
```

## Validate JSON structure

Required sample.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    }
  ]
}
```

Main Python Code:

```
import json

# Load JSON with error handling
try:
    with open("sample.json", "r") as f:
        data = json.load(f)
    print("JSON loaded successfully:", data)
except json.JSONDecodeError as e:
    print("Failed to decode JSON:", e)
```

## Compare two JSON files

Required sample.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    },
    {
      "name": "Bob",
      "last_name": "Johnson",
      "location": ["San Francisco", "Los Angeles"]
    }
  ]
}
```



Required sample\_output.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    },
    {
      "name": "Bob",
      "last_name": "Johnson",
      "location": ["San Francisco", "Los Angeles"]
    },
    {
      "name": "Charlie",
      "last_name": "Brown",
      "location": ["Houston", "Seattle"]
    }
  ]
}
```



Main Python Code:

```
import json

# Load two JSON files
with open("sample.json", "r") as f1, open("sample_output.json", "r") as f2:
    data1 = json.load(f1)
    data2 = json.load(f2)

# Compare the two JSON objects
if data1 == data2:
    print("The JSON files are identical.")
else:
    print("The JSON files are different.")
```

## Append new data to an JSON file

Required sample.json:

```
{
  "people": [
    {
      "name": "Alice",
      "last_name": "Smith",
      "location": ["New York", "Chicago"]
    },
    {
      "name": "Bob",
      "last_name": "Johnson",
      "location": ["San Francisco", "Los Angeles"]
    }
  ]
}
```



Main Python Code:

```
import json

# New data to append
new_person = {
    "name": "Charlie",
    "last_name": "Brown",
    "location": ["Houston", "Seattle"]
}

# Load existing data
with open("sample.json", "r") as f:
    data = json.load(f)

# Append the new person
data["people"].append(new_person)

# Save updated data back to the file
with open("sample.json", "w") as f:
    json.dump(data, f, indent=4)
print("New person added to sample.json.")
```

## Iterate over elements in a JSON list

Sample File (only\_list.json):

```
["apple", "banana", "cherry"]
```

Code

```
import json

with open("only_list.json", "r") as file:
    data = json.load(file)

# Iterate and print each item
for item in data:
    print(item)
```

## Access values by keys in a dictionary

Sample File (only\_dict.json):

```
{"name": "John", "age": 30, "city": "New York"}
```

Code

```
import json
with open("only_dict.json", "r") as file:
    data = json.load(file)
# Access values
print("Name:", data["name"])
print("Age:", data["age"])
print("City:", data["city"])
```



## Nested iteration in a list of lists

Sample File (list\_of\_list.json):

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

Code ↵

```
import json

with open("list_of_list.json", "r") as file:
    data = json.load(file)

# Iterate through outer and inner lists
for sublist in data:
    for item in sublist:
        print(item)
```

## Extract specific fields from a list of dictionaries

Sample File (list\_of\_dict.json):

```
[
  {"id": 1, "name": "Alice"},
  {"id": 2, "name": "Bob"}
]
```

Code ↵

```
import json

with open("list_of_dict.json", "r") as file:
    data = json.load(file)

# Extract and print fields
for item in data:
    print(f"ID: {item['id']}, Name: {item['name']}")
```



## Iterate through lists grouped by a dictionary

Sample File (dict\_of\_list.json):

```
{"fruits": ["apple", "banana"], "vegetables": ["carrot", "spinach"]}
```

Code

```
import json

with open("dict_of_list.json", "r") as file:
    data = json.load(file)

# Iterate through fruits and vegetables
print("Fruits:")

for fruit in data["fruits"]:
    print(fruit)

print("\nVegetables:")

for vegetable in data["vegetables"]:
    print(vegetable)
```

## Access nested dictionaries in a JSON file

Sample File (dict\_of\_dict.json):

```
{
    "person1": {"name": "Alice", "age": 25},
    "person2": {"name": "Bob", "age": 30}
}
```

Code

```
import json

with open("dict_of_dict.json", "r") as file:
    data = json.load(file)

# Access and print nested dictionary details
for person_id, details in data.items():
    print(f"{person_id}: Name - {details['name']}, Age - {details['age']}")
```

# Thank You

